Permitting `static constexpr` variables in `constexpr` functions

Document #:	P2647R0
Date:	2022-09-22
Project:	Programming Language C++
Audience:	EWG

Reply-to:	Barry Revzin <barry.revzin@gmail.com> Jonathan Wakely <cxx@kayari.org>

Contents

§ 1 Introduction

Consider this example:

```
char xdigit(int n) {
  static constexpr char digits[] = "0123456789abcdef";
  return digits[n];
}
```

This function is totally fine. But when we try to extend it to work at compile time too, we run into a problem:

```
constexpr char xdigit(int n) {
  static constexpr char digits[] = "0123456789abcdef";
  return digits[n];
}
```

This is ill-formed. And it's ill-formed for no good reason. We should make it valid.

§ 1.1 History

Originally, no `static` variables could be declared in `constexpr` functions at all. That was relaxed in [P2242R3], which restructured the wording to be based on the actual control flow: rather than the

function body not being allowed to contain a `static` variable declaration, now the rule is that control flow needs to not pass through an initialization of a `static` variable.

This makes sense for the perspective of a `static` (or, worse, `thread_local`) variable, whose initializer could run arbitrary code. But for a `static constexpr` variable, which must, by definition, be constant-initialization, there's no question about whether and when to run what initialization. It's a constant.

§ 1.2 Workarounds

There are several workarounds for getting the above example to work. We could eschew the `static` variable entirely and directly index the literal, but this only works if we need to use it exactly one time:

```
constexpr char xdigit(int n) {
  return "0123456789abcdef"[n];
}
```

We could move the `static` variable into non-local scope, though we wanted to make it local for a reason - it's only relevant to this particular function:

```
static constexpr char digits[] = "0123456789abcdef";
constexpr char xdigit(int n) {
  return digits[n];
}
```

We could make the variable non-`static`, but compilers have difficulty optimizing this, leading to much worse code-gen:

```
constexpr char xdigit(int n) {
  constexpr char digits[] = "0123456789abcdef";
  return digits[n];
}
```

Or we could wrap the variable into a `consteval` lambda, which is the most general workaround for this problem, whose only downside is that… we're writing a `consteval` lambda because we can't write a variable:

```
constexpr char xdigit(int n) {
  auto get_digits = []() consteval { return "0123456789abcdef"; };
  return get_digits()[n];
}
```

Having a local `static constexpr` variable is simply the obvious, most direct solution to the problem. We should permit it.

§ 2 Proposal

Change 7.7 [expr.const]/5.2:

5 — An expression `E` is a *core constant expression* unless the evaluation of `E`, following the rules of the abstract machine ([intro.execution]), would evaluate one of the following:

(5.1) — …

(5.2) — a control flow that passes through a declaration of a `non-constexpr` variable with static ([basic.stc.static]) or thread ([basic.stc.thread]) storage duration;

§ 3 References

[P2242R3] Ville Voutilainen. 2021-07-13. Non-literal variables (and labels and gotos) in constexpr functions.
https://wg21.link/p2242r3